

ENERGIA PARA SOBREVIVER

LISTA DE FIGURAS

Figura 1 - “Onde grafos aparecem?” — Um mosaico com três mini-redes: (i) mapa (cidades/estradas), (ii) rede social (pessoas/conexões), (iii) rede de computadores (roteadores/enlaces), (iv) grafos.....	6
Figura 2 - Grafo simples com vértices A–F e algumas arestas	8
Figura 3 - Não direcionado (a) <i>versus</i> direcionado (b)	9
Figura 4 - <i>In-degree</i> e <i>out-degree</i> com setas visíveis	10
Figura 5 - Caminho simples (a) vs. ciclo (b)	10
Figura 6 - Conexo (a) vs. desconexo (b)	11
Figura 7 - Grafo ponderado com pesos.....	13
Figura 8 - Esperso (a) vs. denso (b)	13
Figura 9 - Matriz de adjacência (0/1)	14
Figura 10 - Lista de adjacência (vetor de listas)	15
Figura 11 - Passo a passo da BFS: Fila e camadas destacadas	19
Figura 12 - Árvore BFS (pais) a partir da origem.....	22
Figura 13 - Passo a passo da DFS: Trilha de exploração e retorno	24
Figura 14 - Grafo com 3 componentes, cada um com uma cor diferente.....	25
Figura 15 - Dijkstra passo a passo: Vértice escolhido em cada iteração e arestas relaxadas.....	27

LISTA DE TABELAS

Tabela 1 - Arestas e pesos (grafo ponderado)	12
Tabela 2 - Comparação: matriz vs. lista (espaço e operações).....	16
Tabela 3 - Comparação de dois caminhos entre os mesmos vértices: número de arestas vs. soma de pesos.....	26
Tabela 4 - Evolução de <code>dist[]</code> e <code>parent[]</code> por iteração	28



LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Matriz de adjacência em C	16
Código-fonte 2 - Lista de adjacência em C	17
Código-fonte 3 - Fila simples (vetor circular ou vetor com índices)	20
Código-fonte 4 - BFS (matriz de adjacência)	21
Código-fonte 5 - BFS com <code>parent[]</code> e reconstrução do caminho	23
Código-fonte 6 - DFS recursiva com lista: Percorre vizinhos diretamente	24
Código-fonte 7 - Dijkstra simples: Seleção do menor <code>dist[]</code> + relaxamento	29
Código-fonte 8 - Função <code>printPath()</code> (reutilizável)	30

SUMÁRIO

GRAFOS E ALGORITMOS DE REDE	6
1 INTRODUÇÃO	6
1.1 Vértices, arestas e adjacência.....	7
1.2 Conceitos fundamentais de grafos	7
1.2.1 Grafo não direcionado e grafo direcionado	8
1.2.2 Grau, <i>in-degree</i> e <i>out-degree</i>	9
1.2.3 Caminho, ciclo e conectividade	10
1.3 Tipos de grafos e modelagem de redes	11
1.4 Representações de grafos na memória (com impacto em desempenho)	14
2 PERCURSOS (BUSCAS) EM GRAFOS	18
2.1 Busca em Largura (BFS).....	18
2.1.1 Estrutura de fila (suporte para BFS)	20
2.1.2 BFS com matriz de adjacência	20
2.1.3 BFS e reconstrução do caminho (<i>parent/edgeTo</i>).....	21
2.2 Busca em Profundidade (DFS).....	23
2.2.1 DFS recursiva (lista de adjacência)	24
2.2.2 Componentes conexas com DFS/BFS	25
3 CAMINHOS E CUSTOS EM REDES	25
3.1 Menor caminho: não ponderado vs. ponderado	26
3.2 Dijkstra (caminho mínimo com pesos não negativos)	27
3.2.1 Implementação em C (sem fila de prioridade)	28
3.2.2 Reconstrução do caminho em Dijkstra	29
3.2.3 Observação importante: Pesos negativos	30
4 CONCLUSÃO.....	30
REFERÊNCIAS.....	32

GRAFOS E ALGORITMOS DE REDE

1 INTRODUÇÃO

Anteriormente, estudamos estruturas que organizam dados em sequências (vetores/listas) e hierarquias (árvores). Porém, muitos problemas reais não são “linha reta” nem “árvore”; eles são **redes**, com vários elementos conectados de diferentes maneiras, como cidades e estradas, pessoas e relações sociais, páginas e links e roteadores e enlaces. Esse padrão aparece tanto que existe uma estrutura criada exatamente para isso: o **grafo**, composto por **vértices** (as entidades) e **arestas** (as conexões) (Sedgewick; Wayne, 2016).

Para tornar o aprendizado mais visual, vamos observar a figura abaixo, que mostra três “mini-redes” (mapa, rede social e rede de computadores). Ela serve para fixar a ideia central: em todos os casos, existe um conjunto de entidades e um conjunto de conexões entre elas. A partir disso, você entende por que grafos aparecem tanto em problemas de “redes” e porque eles são uma das estruturas mais importantes da computação.

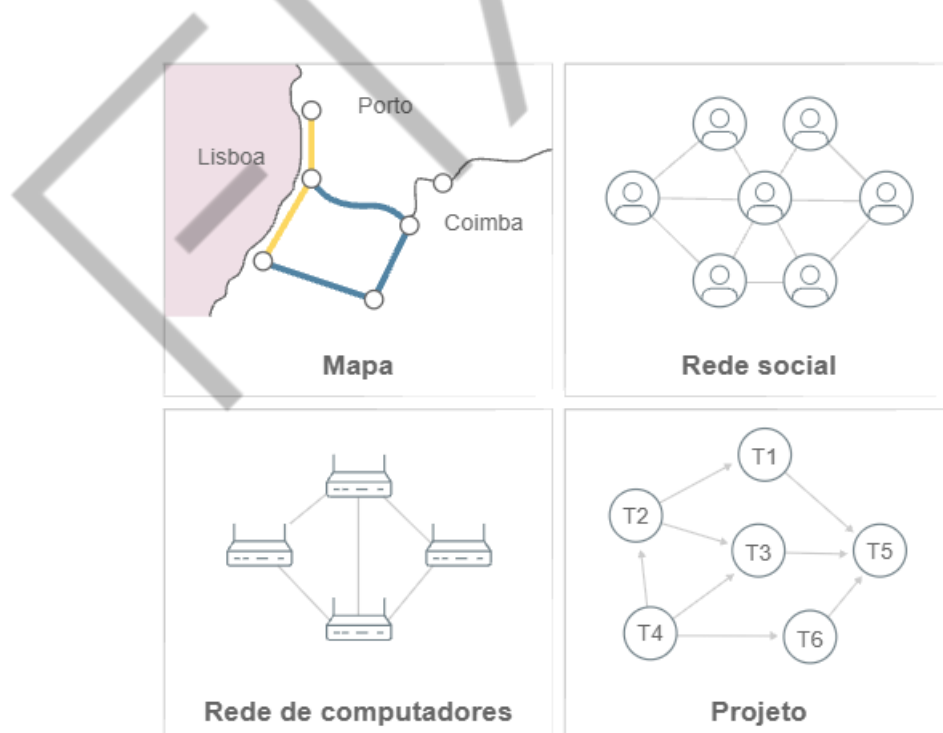


Figura 1 - “Onde grafos aparecem?” — Um mosaico com três mini-redes: (i) mapa (cidades/estradas), (ii) rede social (pessoas/conexões), (iii) rede de computadores (roteadores/enlaces), (iv) grafos
Fonte: Elaborada pelo autor (2026)

Além dos conceitos, esse capítulo também enfatiza como grafos são representados no computador e como isso afeta o desempenho. A escolha entre matriz de adjacência e lista de adjacência muda custos de memória e de operações como “listar vizinhos”, que é uma operação central em BFS e DFS. Materiais clássicos de algoritmos discutem justamente esse tipo de *trade-off* entre representações e custos (MIT Opencourseware, 2011).

1.1 Vértices, arestas e adjacência

Do ponto de vista prático, um vértice representa “algo do mundo” (cidade, pessoa, computador ou tarefa) e uma aresta representa um vínculo entre dois vértices (estrada, amizade, cabo ou dependência). Dizemos que dois vértices são adjacentes quando existe uma aresta ligando um ao outro. Em termos de implementação, “adjacência” vira uma pergunta objetiva: “o vértice v aparece como vizinho de u ?”. É esse questionamento que guia a escolha da estrutura de dados (Sedgewick; Wayne, 2016).

A adjacência é o conceito que conecta o desenho do grafo com a memória do computador. Em uma matriz de adjacência, você responde adjacência consultando uma célula; em uma lista de adjacência, você responde percorrendo uma lista de vizinhos. Esse detalhe, que parece “apenas implementação”, muda o custo do algoritmo inteiro quando o grafo cresce (Shukla, 2023; Bockholt; Black, 2021).

1.2 Conceitos fundamentais de grafos

Um grafo pode ser descrito como um conjunto de vértices e um conjunto de arestas que conectam esses vértices. Em termos formais, um grafo é um par (V, E) , em que V é o conjunto de vértices e E é o conjunto de arestas (Black, 2007). Em um contexto prático, você pode pensar em V como “quem participa da rede” e E como “quais conexões existem”.

Para ajudar na visualização, a figura a seguir apresenta um grafo pequeno com conexões suficientes para mostrar “vizinhança” e “grau”.

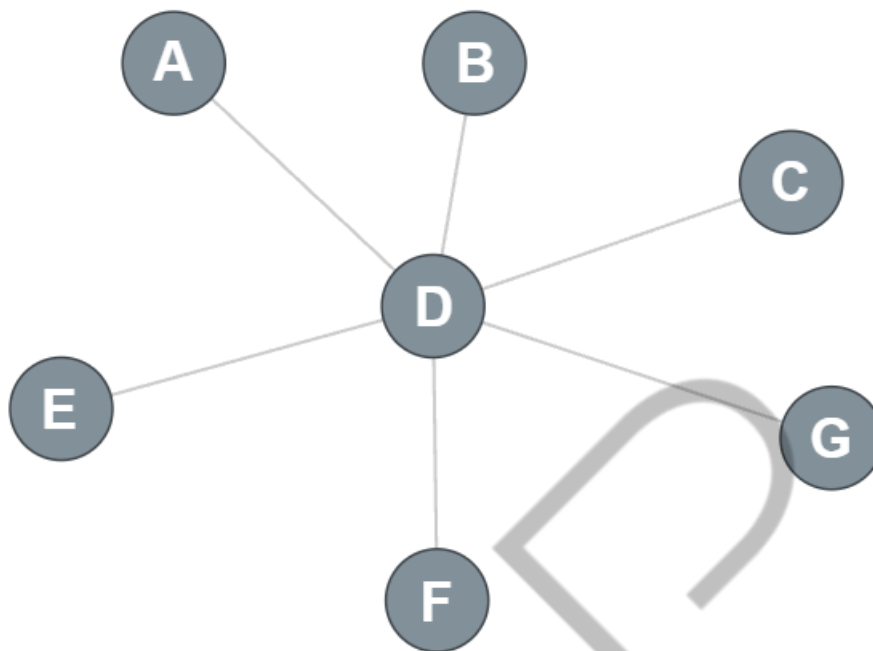


Figura 2 - Grafo simples com vértices A–F e algumas arestas
Fonte: Elaborada pelo autor (2026)

A partir dessa imagem, introduzimos o termo **adjacência**: dois vértices são adjacentes se existe uma aresta ligando-os. Essa ideia é essencial porque muitas operações em grafos são do tipo “quais são os vizinhos desse vértice?”; e isso aparece diretamente na implementação, tanto em matriz quanto em lista de adjacência.

1.2.1 Grafo não direcionado e grafo direcionado

A figura abaixo mostra, lado a lado, um grafo não direcionado e um grafo direcionado (dígrafo) usando o mesmo conjunto de vértices, para facilitar a comparação. No grafo **não direcionado**, as arestas não têm setas: a conexão é bidirecional. Formalmente, as arestas são pares **não ordenados** $\{u, v\}$ (Black, 2007). Isso significa que a aresta $\{A, B\}$ representa uma ligação entre A e B sem distinguir “ida” e “volta”.

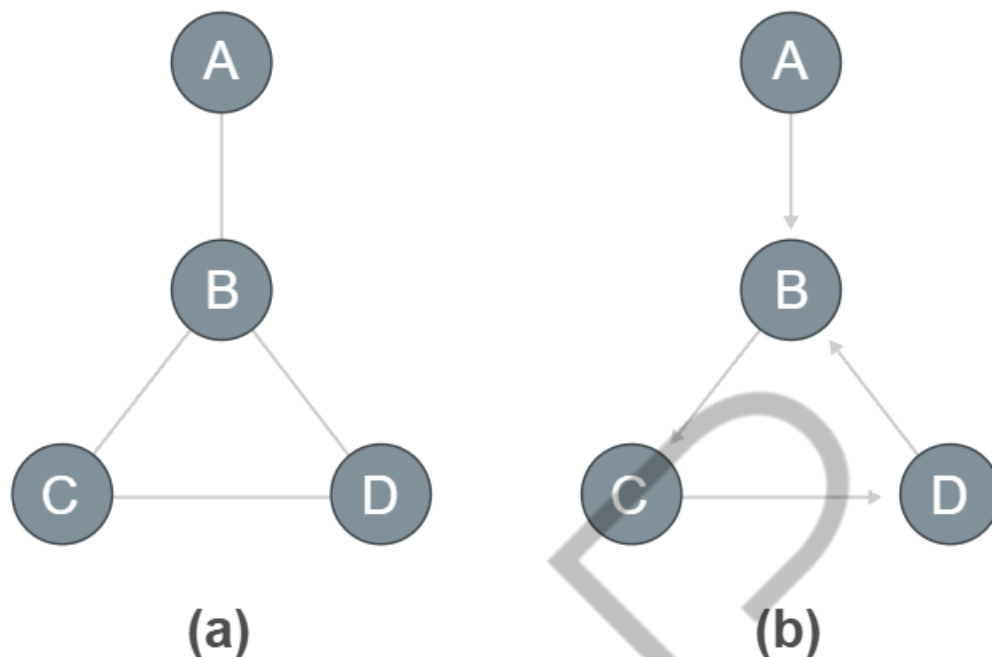


Figura 3 - Não direcionado (a) *versus* direcionado (b)
Fonte: Elaborada pelo autor (2026)

Já no grafo **direcionado**, as arestas têm direção e são pares **ordenados** (u, v). A imagem apresenta essa funcionalidade com setas visíveis: $A \rightarrow B$ não implica $B \rightarrow A$ (Black, 2023). Essa diferença é crítica em problemas de dependência (uma tarefa depende de outra) ou navegação (*links* na web), em que o sentido das conexões importa.

Na implementação, essa distinção aparece de modo direto: em um grafo não direcionado, inserir a aresta $u - v$ significa registrar v como vizinho de u e u como vizinho de v . Em um dígrafo, inserir $u \rightarrow v$ registra apenas a saída de u para v . Esse detalhe será importante quando implementarmos `addEdge()` na lista de adjacência.

1.2.2 Grau, *in-degree* e *out-degree*

A figura abaixo mostra um vértice central com setas entrando e saindo. A partir dela, explicamos que, no grafo não direcionado, o **grau** de um vértice é o número de arestas conectadas a ele. Já em grafos direcionados, usamos duas medidas: **grau de entrada (*in-degree*)** e **grau de saída (*out-degree*)**. Isso é particularmente intuitivo em redes sociais: “quantos seguem você” (entradas) *versus* “quantos você segue” (saídas).

Aqui o objetivo não é “decorar nomes”, mas conectar a ideia e a leitura do grafo: é necessário olhar para a figura e conseguir contar entradas/saídas sem confusão. Em seguida, reforçamos que muitas análises de redes (como identificar nós influentes) começam por medidas simples como grau.

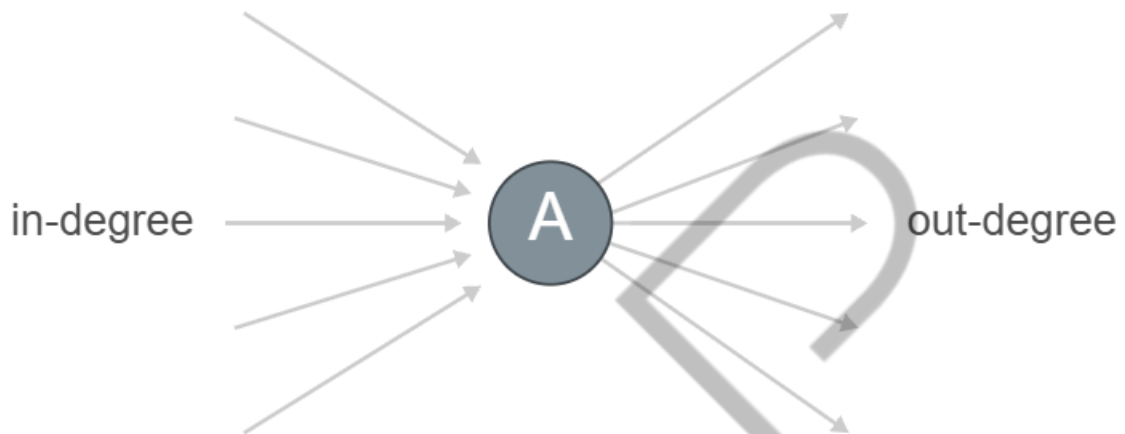


Figura 4 - *In-degree* e *out-degree* com setas visíveis
Fonte: Elaborada pelo autor (2026)

1.2.3 Caminho, ciclo e conectividade

Um **caminho** é uma sequência de vértices ligados por arestas consecutivas e um **ciclo** ocorre quando o caminho “fecha” e retorna ao ponto inicial.

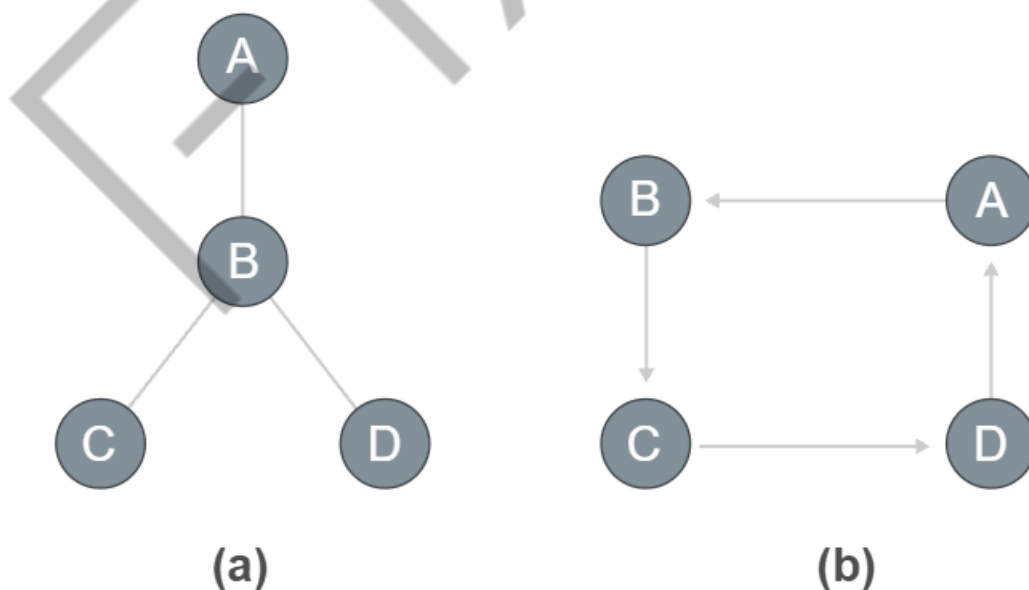


Figura 5 - Caminho simples (a) vs. ciclo (b)
Fonte: Elaborada pelo autor (2026)

A conectividade é outro conceito essencial. A figura “**Conexo (a) vs. desconexo (b)**” compara um grafo conexo e um desconexo. No grafo desconexo, existe mais de um grupo isolado, chamado **componente conexa**. Isso é muito útil em redes: se uma rede de computadores tem duas componentes, significa que existem máquinas que não conseguem se comunicar com outras.

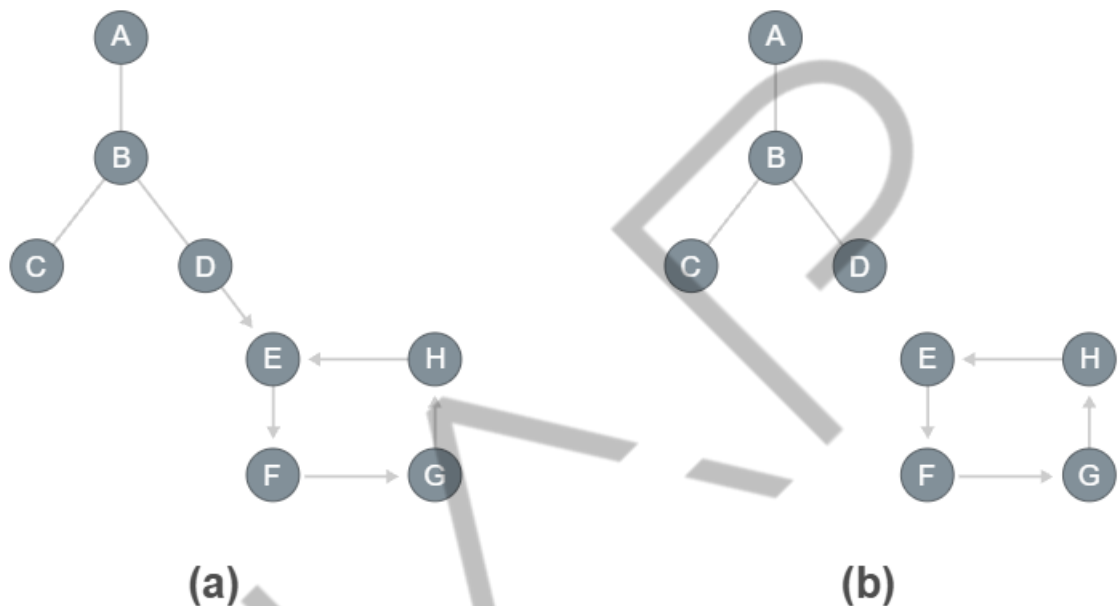


Figura 6 - Conexo (a) vs. desconexo (b)
Fonte: Elaborada pelo autor (2026)

Esse tópico é importante para BFS/DFS, porque percursos são justamente “formas sistemáticas” de descobrir quais vértices podem ser alcançados e quais pertencem à mesma componente.

1.3 Tipos de grafos e modelagem de redes

Para modelar corretamente uma rede, três perguntas resolvem grande parte do caminho:

1. Arestas têm direção? (sim → dígrafo; não → grafo não direcionado);
2. Arestas têm custo/peso? (sim → grafo ponderado);
3. O grafo é esparsos ou denso? (impacta representação).

Essa “triagem” é importante porque as operações e algoritmos mudam: por exemplo, BFS encontra menor caminho em número de arestas (não ponderado),

enquanto Dijkstra trata menor custo total (ponderado, pesos não negativos) (Black, 2020; Sedgewick; Wayne, 2016).

Primeiro, vamos discutir grafos **ponderados**. Uma aresta pode carregar um peso que representa distância, tempo ou custo; isso é comum em redes de transporte e roteamento. A tabela “Arestas e pesos (grafo ponderado)” apresenta um conjunto de arestas com pesos. Em seguida, a figura “**Grafo ponderado com pesos**” mostra o mesmo grafo com os pesos escritos nas arestas. A função dessas duas inserções é clara: você enxerga que “menor caminho” pode significar **menos custo**, não apenas menos passos.

Aresta (u, v)	Peso (w)
(A, D)	7
(B, D)	3
(C, D)	5
(D, E)	4
(D, F)	6
(D, G)	2
(E, F)	1
(F, G)	8

Tabela 1 - Arestas e pesos (grafo ponderado)
Fonte: Elaborada pelo autor (2026)

Os Grafos que Modelam as Rotas de Distribuição de Energia

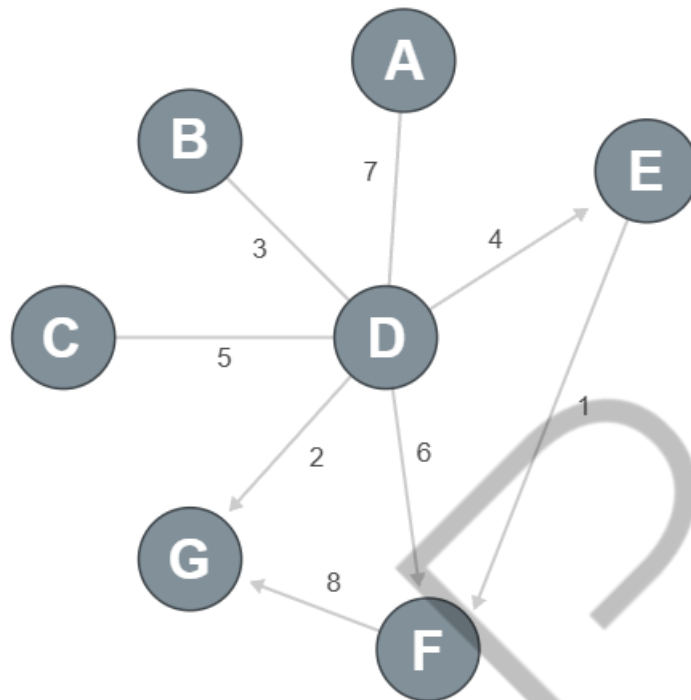


Figura 7 - Grafo ponderado com pesos
Fonte: Elaborada pelo autor (2026)

Agora, vamos entender os grafos **densos** e **esparso**. Aqui, a figura “**Esparso (a) vs. denso (b)**” (dois grafos com o mesmo número de vértices, um com poucas arestas e outro com muitas) é crucial, pois conecta diretamente com o próximo conteúdo: grafos esparsos tendem a ser melhor representados por listas; já os grafos densos podem se beneficiar de matrizes em certas operações (MIT Opencourseware, 2011).

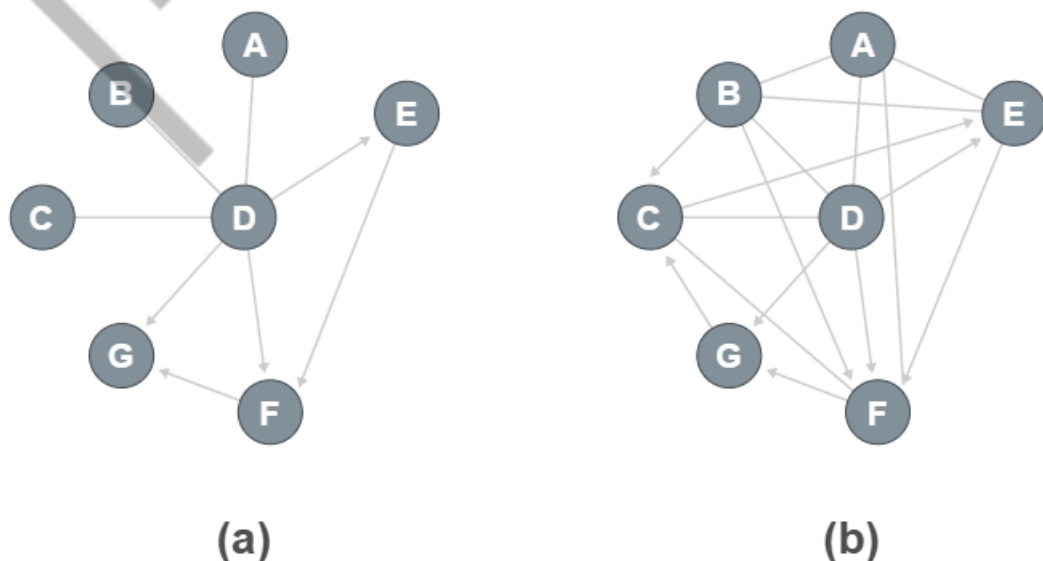


Figura 8 - Esparso (a) vs. denso (b)
Fonte: Elaborada pelo autor (2026)

1.4 Representações de grafos na memória (com impacto em desempenho)

Aqui vamos aprender duas formas consagradas de armazenar grafos:

- **Matriz de adjacência:** uma matriz $V \times V$ com 0/1 (ou peso);
- **Lista de adjacência:** um vetor de listas (cada lista guarda os vizinhos).

A escolha entre matriz e lista não é “apenas estilo”: ela afeta o consumo de memória e o custo de operações centrais, especialmente “listar vizinhos” (Shukla, 2023; Bockholt; Black, 2021).

A figura “Matriz de adjacência (0/1)” mostra como a matriz funciona: $mat[u][v] = 1$ indica aresta de u para v . O guia do olhar aqui é: escolha um vértice i e leia a linha i para descobrir para onde ele aponta ou escolha uma coluna para ver quem aponta para ele. Em grafos não direcionados, a matriz tende a ser simétrica, pois a ligação vale nos dois sentidos (Shukla, 2023).

	1	2	3	4	5
1	0	1	1	0	0
2	1	0	0	1	0
3	1	0	0	1	1
4	0	1	1	0	0
5	0	0	1	0	0

Figura 9 - Matriz de adjacência (0/1)
Fonte: Elaborada pelo autor (2026)

Em seguida, vamos entender a lista de adjacência na figura “Lista de adjacência (vetor de listas)”, em que cada vértice aponta para uma lista de vizinhos. Na lista do vértice 2 aparecem os vértices 0 e 4, então 2 se conecta com 0 e 4. Essa representação é especialmente compacta quando o grafo é esparsos, pois armazena apenas as conexões existentes (Bockholt; Black, 2021).

Lista de Adjacência (vetor de listas)

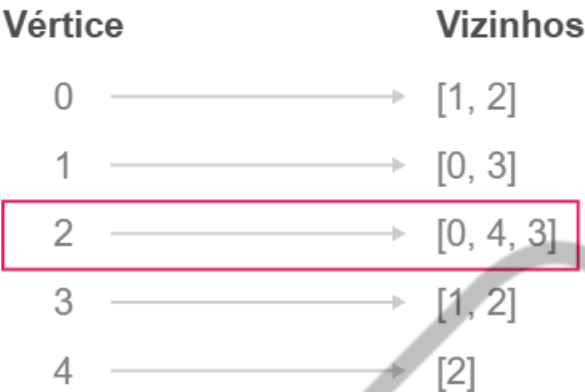


Figura 10 - Lista de adjacência (vetor de listas)
Fonte: Elaborada pelo autor (2026)

Por fim, a tabela abaixo resume os *trade-offs* mais importantes: verificar aresta é $O(1)$ na matriz, mas listar vizinhos custa $O(V)$; já na lista, listar vizinhos é proporcional ao número de vizinhos. Esse ponto é fundamental porque BFS/DFS “vivem” de visitar vizinhos. O próprio material do MIT destaca que BFS pode ficar $O(V^2)$ usando matriz por causa da varredura de vizinhos (MIT Opencourseware, 2011).

Critério/Operação	Matriz de adjacência	Lista de adjacência
Espaço	$O(V^2)$	$O(V + E)$
Verificar se existe aresta (u, v)	$O(1)$ → basta checar $mat[u][v]$	$O(\text{grau}(u))$ (ou $O(E)$ no pior caso) → buscar v na lista de u
Listar vizinhos de u	$O(V)$ → varre a linha/coluna de u	$O(\text{grau}(u))$ → percorre só os vizinhos existentes
Inserir aresta (u, v)	$O(1)$	$O(1)$ (inserção no início da lista, podendo variar se quiser manter ordenado)
Remover aresta (u, v)	$O(1)$	$O(\text{grau}(u))$ (precisa localizar v na lista)

Critério/Operação	Matriz de adjacência	Lista de adjacência
BFS/DFS (impacto prático)	Pode induzir varredura $O(V)$ por vértice $\Rightarrow$ $O(V^2)$ em grafos esparsos	Visita vizinhos diretamente $\Rightarrow$ $O(V + E)$

Tabela 2 - Comparação: matriz vs. lista (espaço e operações)
Fonte: Elaborada pelo autor (2026)

O código-fonte a seguir cria a base para representar conexões: `adj[u][v]=1` indica que existe uma aresta. Em grafos não direcionados, adicionamos a conexão nos dois sentidos, o que explica a simetria mostrada na figura “**Não direcionado (a) versus direcionado (b)**”.

```
#include <stdio.h>

#define MAXV 50

// Inicializa a matriz adj com zeros: sem arestas registradas
void initMatrix(int adj[MAXV][MAXV], int V) {
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            adj[i][j] = 0;
        }
    }
}

// Adiciona aresta u -> v (grafo direcionado)
void addEdgeDirected(int adj[MAXV][MAXV], int u, int v) {
    adj[u][v] = 1;
}

// Adiciona aresta u - v (grafo não direcionado)
void addEdgeUndirected(int adj[MAXV][MAXV], int u, int v)
{
    adj[u][v] = 1;
    adj[v][u] = 1;
}
```

Código-fonte 1 - Matriz de adjacência em C
Fonte: Elaborado pelo autor (2026)

O código-fonte “**Lista de adjacência em C**” implementa a ideia visual da figura “**Esparso (a) vs. denso (b)**”: cada posição do vetor **adj** corresponde a um vértice, e ali

Os Grafos que Modelam as Rotas de Distribuição de Energia

guardamos uma lista de vizinhos. Em grafos não direcionados, inserimos dois nós ($u \rightarrow v$ e $v \rightarrow u$), como já discutido anteriormente.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int v;                // vizinho
    struct Node* next;    // próximo vizinho na lista
} Node;

typedef struct Graph {
    int V;                // número de vértices
    Node** adj;           // vetor de listas de adjacência
} Graph;

Node* createNode(int v) {
    Node* n = (Node*) malloc(sizeof(Node));
    n->v = v;
    n->next = NULL;
    return n;
}

Graph* createGraph(int V) {
    Graph* g = (Graph*) malloc(sizeof(Graph));
    g->V = V;
    g->adj = (Node**) malloc(V * sizeof(Node*));

    for (int i = 0; i < V; i++) {
        g->adj[i] = NULL; // listas inicialmente vazias
    }
    return g;
}

// Insere v como vizinho de u (u -> v)
void addEdgeDirectedList(Graph* g, int u, int v) {
    Node* n = createNode(v);
    n->next = g->adj[u];
    g->adj[u] = n;
}

// Para grafo não direcionado: insere nos dois sentidos (u-
v)
void addEdgeUndirectedList(Graph* g, int u, int v) {
    addEdgeDirectedList(g, u, v);
    addEdgeDirectedList(g, v, u);
}
```

Código-fonte 2 - Lista de adjacência em C

Fonte: Elaborado pelo autor (2026)

A representação por **listas de adjacência**, implementada no código anterior, não é apenas uma forma “diferente” de guardar um grafo: ela define a maneira mais direta de executar os algoritmos de percurso que veremos a seguir. Isso acontece porque, nessa representação, para cada vértice u , a estrutura $g \rightarrow \text{adj}[u]$ contém exatamente o conjunto de vértices **adjacentes** (vizinhos) de u , isto é, o “leque de saída” do vértice, aquilo que pode ser explorado a partir dele em um algoritmo de busca (IME-USP, 2026). Em outras palavras, quando um algoritmo pergunta “quais vizinhos eu consigo alcançar a partir de u ?”, a lista de adjacência já entrega a resposta pronta, sem precisar varrer possibilidades que não existem.

Essa característica se conecta diretamente ao modo como os percursos em grafos são descritos em abstrações clássicas: a operação central é **iterar sobre todos os vértices adjacentes a um vértice dado** (por exemplo, uma função $\text{adj}(u)$) e, a partir disso, construir BFS, DFS e outras rotinas de processamento de grafos (Sedgewick; Wayne, 2016). Portanto, ao dominar a leitura e a construção da lista de adjacência, você já preparou o “motor” que os algoritmos usarão: visitar um vértice e, em seguida, percorrer seus vizinhos.

Com esse ponto consolidado, avançamos para a **Busca em Largura (BFS)**. A BFS é um algoritmo que explora primeiro os vizinhos mais próximos antes de avançar para níveis mais distantes, e por isso ela depende de duas ideias simples: (1) **percorrer os vizinhos** de cada vértice (o que a lista de adjacência facilita) e (2) controlar a ordem de exploração com uma **fila**, garantindo que os vértices descobertos primeiro sejam processados primeiro (Black, 2019).

2 PERCURSOS (BUSCAS) EM GRAFOS

2.1 Busca em Largura (BFS)

A Busca em Largura (BFS, do inglês *Breath First Search*) é uma estratégia de percurso que, ao partir de um vértice origem, visita primeiro todos os vértices a **uma aresta** de distância, depois os vértices a **duas arestas** e assim por diante; por isso, ela é frequentemente descrita como uma exploração “por camadas”. Formalmente,

BFS é uma busca que considera os **vizinhos de um vértice** antes de seguir para vizinhos mais distantes, e sua implementação típica utiliza uma **fila (queue)** para manter a ordem correta de processamento (Black, 2019).

Além disso, a BFS precisa, repetidamente, escolher um vértice u para processar e então **iterar sobre sua lista de adjacência**, examinando cada vizinho v . Em uma implementação baseada em lista de adjacência, essa etapa é natural: basta percorrer os nós armazenados em $\text{adj}[u]$. Esse padrão (retirar um vértice da fila $\rightarrow$ percorrer sua adjacência $\rightarrow$ enfileirar vizinhos não visitados) é exatamente a mecânica operacional da BFS e também o motivo pelo qual ela constrói, junto com as distâncias, uma “árvore de descoberta” com predecessores (Runestone Academy, 2026).

Do ponto de vista de projeto de algoritmos, essa forma de escrita também reforça uma ideia importante: boa parte dos algoritmos de grafos pode ser construída sobre uma abstração mínima: “para cada vértice, consiga iterar seus adjacentes”. Materiais clássicos enfatizam que essa operação ($\text{adj}()$) é tão central que muitos algoritmos podem ser desenvolvidos a partir dela (Sedgewick; Wayne, 2016). Assim, ao observar a figura a seguir e, na sequência, o código de BFS, repare que a lista de adjacência define “quem eu posso visitar a partir daqui”, enquanto a fila define “em que ordem eu devo visitar”.

Busca em Largura (BFS)

Ordem: 0, 1, 2, 3, 4, 5, 6

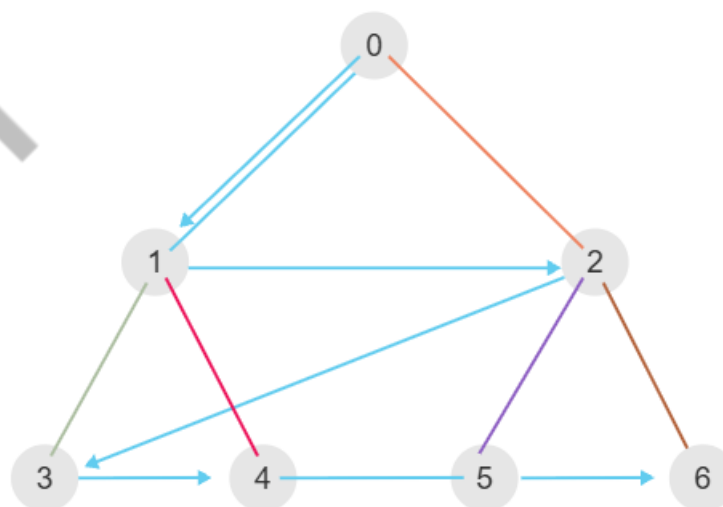


Figura 11 - Passo a passo da BFS: Fila e camadas destacadas
Fonte: Elaborada pelo autor (2026)

2.1.1 Estrutura de fila (suporte para BFS)

Antes de programar BFS, precisamos de uma fila. A fila representa o conjunto de vértices “descobertos, mas ainda não processados”. Ao implementar BFS, você verá que o algoritmo se resume a um ciclo: remove um vértice, percorre seus vizinhos e enfileira os vizinhos ainda não visitados (MIT Opencourseware, 2011).

O código a seguir não é BFS ainda; ele é a infraestrutura que permite escrever BFS de forma clara. Separar fila e BFS ajuda iniciantes, já que reduz a quantidade de ideias novas ao mesmo tempo.

```
#define QMAX 1000

typedef struct {
    int data[QMAX];
    int front, rear;
} Queue;

void qInit(Queue *q) { q->front = 0; q->rear = 0; }
int qEmpty(Queue *q) { return q->front == q->rear; }

void qPush(Queue *q, int x) { q->data[q->rear++] = x; }
int qPop(Queue *q) { return q->data[q->front++]; }
```

Código-fonte 3 - Fila simples (vetor circular ou vetor com índices)
Fonte: Elaborado pelo autor (2026)

2.1.2 BFS com matriz de adjacência

Na matriz, “achar vizinhos de u” significa varrer toda a linha `adj[u][v]` para `v` de 0 a `V-1`. Isso torna o código simples, mas pode ser custoso se `V` for grande e o grafo for esparso (Sedgewick; Wayne, 2016).

O código abaixo mostra o núcleo da BFS: marcar visitados, usar fila e expandir vizinhos. O ponto central é observar o `for (v=0; v<V; v++)`: ele existe porque a matriz não guarda “a lista de vizinhos pronta”; ela guarda uma tabela completa de possibilidades.

```
#include <stdio.h>

void bfsMatrix(int adj[MAXV][MAXV], int V, int s) {
```

```
int visited[MAXV] = {0};
Queue q; qInit(&q);

visited[s] = 1;      // marca origem
qPush(&q, s);       // enfileira origem

while (!qEmpty(&q)) {
    int u = qPop(&q);
    printf("Visitou: %d\n", u);

    // varre toda a linha para encontrar vizinhos
    for (int v = 0; v < V; v++) {
        if (adj[u][v] == 1 && !visited[v]) {
            visited[v] = 1;
            qPush(&q, v);
        }
    }
}
```

Código-fonte 4 - BFS (matriz de adjacência)
Fonte: Elaborado pelo autor (2026)

2.1.3 BFS e reconstrução do caminho (*parent/edgeTo*)

Além de visitar, frequentemente queremos **reconstruir o caminho** da origem até um destino. A ideia clássica é salvar, para cada vértice v , “de onde eu vim quando descobri v pela primeira vez”. Isso forma uma árvore enraizada na origem e permite reconstruir um caminho retrocedendo pelos pais (Sedgewick; Wayne, 2016).

A figura “**Árvore BFS (pais) a partir da origem**” mostra que BFS não só visita: ela constrói uma árvore de descoberta. Cada aresta dessa árvore aponta para o “pai” do vértice, que é exatamente o que guardamos no vetor `parent[]`.

Os Grafos que Modelam as Rotas de Distribuição de Energia

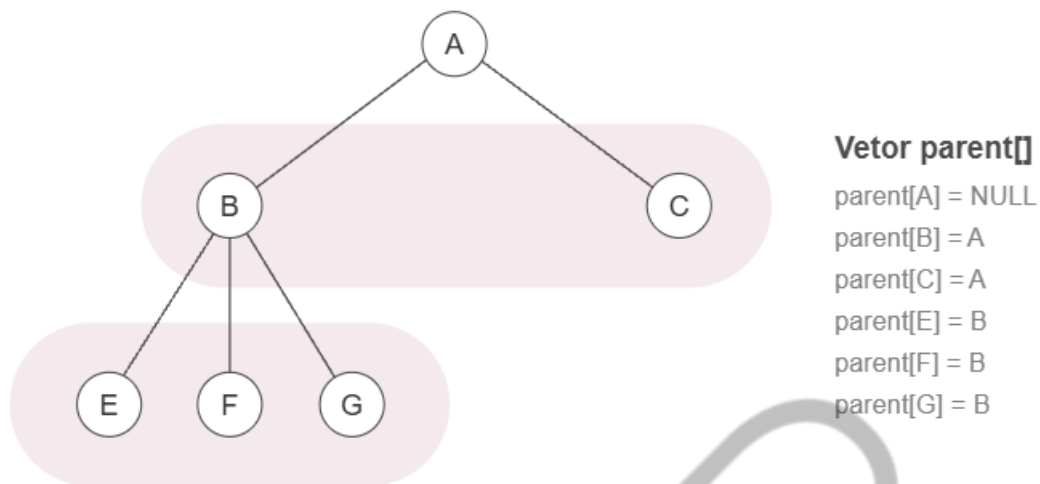


Figura 12 - Árvore BFS (pais) a partir da origem
Fonte: Elaborada pelo autor (2026)

No próximo código, `parent[v] = u` significa que “para chegar em `v`, eu passei por `u`”. Ao final, reconstruir o caminho vira um processo simples: começar em `t`, ir para `parent[t]` e assim por diante até `s`.

```
#include <stdio.h>

void printPath(int parent[], int s, int t) {
    if (t == -1) { printf("Sem caminho.\n"); return; }

    // reconstrução reversa em um vetor auxiliar
    int stack[MAXV], top = 0;
    for (int v = t; v != -1; v = parent[v]) {
        stack[top++] = v;
        if (v == s) break;
    }

    if (stack[top-1] != s) { printf("Sem caminho.\n"); return; }

    // imprime do início ao fim
    for (int i = top - 1; i >= 0; i--) {
        printf("%d%s", stack[i], (i ? " -> " : "\n"));
    }
}

void bfsMatrixWithParent(int adj[MAXV][MAXV], int V, int s,
int parent[]) {
    int visited[MAXV] = {0};
    Queue q; qInit(&q);

    for (int i = 0; i < V; i++) parent[i] = -1;
```

```
visited[s] = 1;
qPush(&q, s);

while (!qEmpty(&q)) {
    int u = qPop(&q);

    for (int v = 0; v < V; v++) {
        if (adj[u][v] == 1 && !visited[v]) {
            visited[v] = 1;
            parent[v] = u;    // salva o pai no momento
                               da descoberta
            qPush(&q, v);
        }
    }
}
```

Código-fonte 5 - BFS com parent[] e reconstrução do caminho
Fonte: Elaborado pelo autor (2026)

2.2 Busca em Profundidade (DFS)

A Busca em Profundidade (DFS, do inglês *Depth First Search*) segue uma estratégia diferente: ela tenta avançar o máximo possível em uma direção antes de voltar e explorar alternativas. Em termos práticos, é como explorar um labirinto sempre entrando na primeira porta disponível; quando não há mais portas, você volta (*backtracking*) (Sedgewick; Wayne, 2016).

A figura “**Passo a passo da DFS: Trilha de exploração e retorno**” evidencia dois momentos: (i) o avanço “em profundidade” e (ii) o retorno quando não há vizinhos não visitados. Esse retorno explica por que a recursão combina bem com DFS.

Busca em Profundidade (DFS)

Ordem: 0, 1, 3, 4, 2, 5, 6

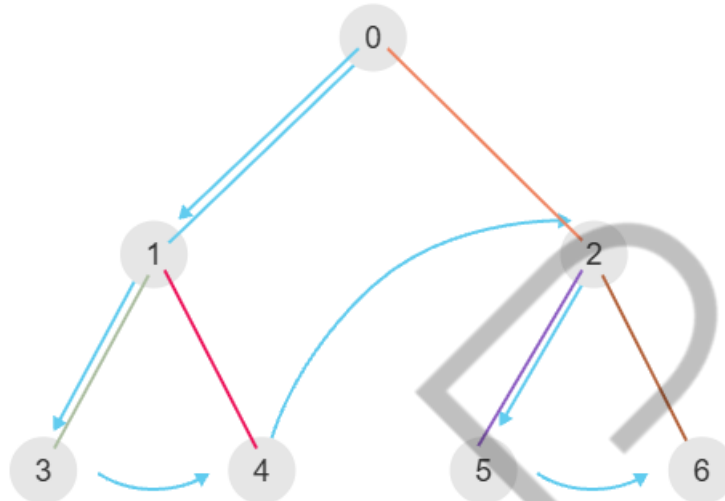


Figura 13 - Passo a passo da DFS: Trilha de exploração e retorno
Fonte: Elaborada pelo autor (2026)

2.2.1 DFS recursiva (lista de adjacência)

A implementação recursiva segue três passos simples: marcar o vértice como visitado, processá-lo e chamar DFS para cada vizinho ainda não visitado. O ponto mais importante é entender o papel de `visited[]`: grafos podem ter ciclos e, sem marcação, o algoritmo poderia repetir infinitamente (Sedgewick; Wayne, 2016).

O código abaixo combina bem com a lista de adjacência, pois iterar vizinhos é apenas percorrer a lista `adj[u]`.

```
#include <stdio.h>

void dfsList(Graph *g, int u, int visited[]) {
    visited[u] = 1;
    printf("Visitou: %d\n", u);

    for (Node *p = g->adj[u]; p != NULL; p = p->next) {
        int v = p->v;
        if (!visited[v]) {
            dfsList(g, v, visited);
        }
    }
}
```

Código-fonte 6 - DFS recursiva com lista: Percorre vizinhos diretamente
Fonte: Elaborado pelo autor (2026)

2.2.2 Componentes conexas com DFS/BFS

Uma aplicação direta de DFS (ou BFS) é encontrar **componentes**: basta iniciar o percurso em um vértice ainda não visitado; tudo que ele alcançar pertence à mesma componente. Repetindo isso para cada vértice não visitado, contamos quantas componentes existem e quais vértices pertencem a cada uma (Sedgewick; Wayne, 2016).

A figura abaixo mostra que, mesmo com o mesmo número de vértices, a rede pode estar “quebrada” em grupos. O algoritmo não “adivinha”; ele descobre ao percorrer.

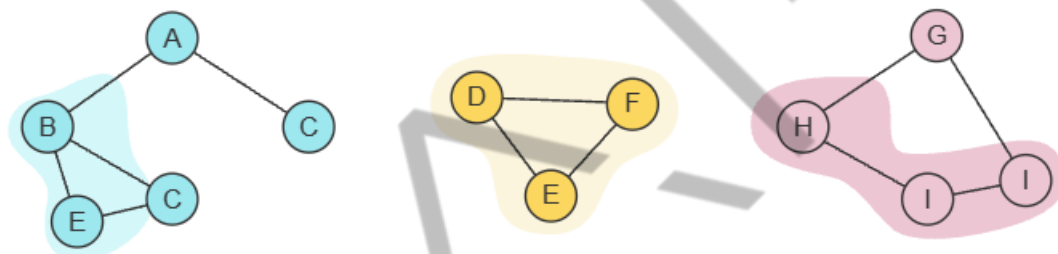


Figura 14 - Grafo com 3 componentes, cada um com uma cor diferente
Fonte: Elaborada pelo autor (2026)

3 CAMINHOS E CUSTOS EM REDES

Em BFS, aprendemos a percorrer um grafo **por camadas**, visitando primeiro os vértices mais próximos da origem em **número de arestas**. Essa estratégia não serve apenas para “visitar todos os nós”; ela também fundamenta a noção de **menor caminho em grafos não ponderados**, em que “mais curto” significa “com menos passos (arestas)”. Em outras palavras, quando todas as conexões têm o mesmo “custo”, a BFS encontra a menor distância (em quantidade de arestas) da origem para os demais vértices (Sedgewick; Wayne, 2016).

Entretanto, redes reais raramente têm conexões equivalentes. Em mapas, cada estrada tem um comprimento; em redes de computadores, um enlace pode ter custo associado a atraso, largura de banda ou outra métrica; em logística, rotas têm preço e tempo diferentes. Nesses casos, o problema muda: o “melhor” caminho passa a ser aquele com **menor soma de pesos**, e um caminho com menos arestas pode ser mais

caro do que outro com mais. Essa é a virada conceitual: saímos do “menor número de passos” (BFS) e entramos no “menor custo total” (caminho mínimo em grafo ponderado) (Sedgewick; Wayne, 2016).

Para grafos ponderados com **pesos não negativos**, o algoritmo clássico para encontrar caminhos mínimos a partir de uma origem é o **Dijkstra**, que constrói incrementalmente uma **árvore de caminhos mínimos** ao sempre escolher o próximo vértice com menor custo conhecido e relaxar suas arestas (Sedgewick; Wayne, 2016). Esse princípio é tão importante que aparece explicitamente em protocolos de roteamento do tipo *link-state*: no OSPF, por exemplo, o documento do padrão descreve a primeira etapa do procedimento como o algoritmo de Dijkstra para montar a árvore de menor custo (*shortest-path tree*) com base nos custos de enlace (MOY, 1998).

3.1 Menor caminho: não ponderado vs. ponderado

O problema de menor caminho é um dos modelos mais úteis em redes.

Importante: “menor” depende do tipo de grafo. Em grafo não ponderado, “menor” costuma significar “menor número de arestas”. Em grafo ponderado, “menor” significa “menor soma de pesos” (Black, 2020).

A tabela a seguir apresenta a seguinte situação: um caminho pode ter 2 arestas e custo 100, enquanto outro tem 4 arestas e custo 12. Isso mostra por que BFS resolve bem o caso não ponderado, mas falha no caso ponderado (Black, 2020).

Caminho	Sequência de vértices	Nº de arestas	Soma dos pesos (custo)
P1 (menos arestas)	A → D → G	2	100
P2 (menor custo)	A → B → C → E → G	4	12

Tabela 3 - Comparação de dois caminhos entre os mesmos vértices: número de arestas vs. soma de pesos
Fonte: Elaborada pelo autor (2026)

3.2 Dijkstra (caminho mínimo com pesos não negativos)

Dijkstra é um algoritmo clássico de caminho mínimo a partir de uma origem em grafos ponderados com pesos não negativos. A ideia é manter `dist[]` (melhor custo conhecido) e ir “confirmando” vértices do menor para o maior custo, relaxando arestas: se passar por `u` melhora o custo até `v`, atualizamos `dist[v]` e registramos o predecessor para reconstruir o caminho (Sedgewick; Wayne, 2025).

A figura evidencia o comportamento incremental do algoritmo de Dijkstra: a cada iteração, escolhemos o vértice ainda não finalizado com o menor valor de `dist[]`. Em seguida, realizamos o relaxamento das arestas que saem desse vértice, tentando melhorar (diminuir) as distâncias dos seus vizinhos. Note que os valores de `dist[]` só diminuem quando encontramos um caminho melhor durante o relaxamento; caso contrário, permanecem iguais. Esse processo se repete até que todos os vértices alcançáveis sejam finalizados.

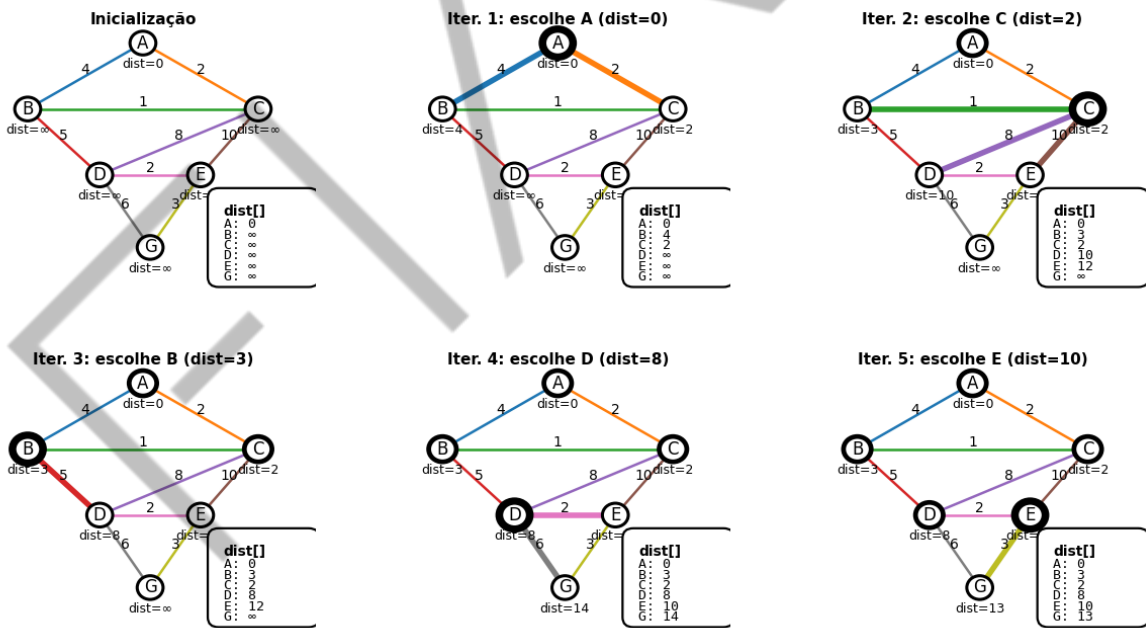


Figura 15 - Dijkstra passo a passo: Vértice escolhido em cada iteração e arestas relaxadas
Fonte: Elaborada pelo autor (2026)

Passo	Vértice escolhido	Atualizações (relaxamento)	dist[A,B,C,D,E,G]	parent[A,B,C,D,E,G]
0	— (init)	—	[0, ∞, ∞, ∞, ∞, ∞]	[NULL, NULL, NULL, NULL, NULL, NULL]

Passo	Vértice escolhido	Atualizações (relaxamento)	dist[A,B,C,D,E,G]	parent[A,B,C,D,E,G]
1	A	B $\leftarrow$ 4 (pai=A); C $\leftarrow$ 2 (pai=A)	[0, 4, 2, ∞ , ∞ , ∞]	[NULL, A, A, NULL, NULL, NULL]
2	C	B: 4 $\rightarrow$ 3 (pai=C); D $\leftarrow$ 10 (pai=C); E $\leftarrow$ 12 (pai=C)	[0, 3, 2, 10, 12, ∞]	[NULL, C, A, C, C, NULL]
3	B	D: 10 $\rightarrow$ 8 (pai=B)	[0, 3, 2, 8, 12, ∞]	[NULL, C, A, B, C, NULL]
4	D	E: 12 $\rightarrow$ 10 (pai=D); G $\leftarrow$ 14 (pai=D)	[0, 3, 2, 8, 10, 14]	[NULL, C, A, B, D, D]
5	E	G: 14 $\rightarrow$ 13 (pai=E)	[0, 3, 2, 8, 10, 13]	[NULL, C, A, B, D, E]

Tabela 4 - Evolução de `dist[]` e `parent[]` por iteração

Fonte: Elaborada pelo autor (2026)

3.2.1 Implementação em C (sem fila de prioridade)

A versão mais eficiente usa fila de prioridade, mas, para aprendizado, é muito comum começar com a versão didática $O(V^2)$, que escolhe o menor `dist[]` por varredura. Isso mantém o foco na lógica do algoritmo e no relaxamento (Sedgewick; Wayne, 2016).

No código a seguir, repare em duas partes: (i) a função que encontra o vértice não usado com menor `dist[]` e (ii) o laço de relaxamento, que tenta melhorar `dist[v]` usando `dist[u] + w[u][v]`. Esse é o coração do Dijkstra (Sedgewick; Wayne, 2016).

```
#include <stdio.h>

#define INF 1000000000

// Retorna o índice do vértice não finalizado com menor
dist
int minDist(int dist[], int used[], int V) {
    int best = INF, idx = -1;
```

```
        for (int i = 0; i < V; i++) {
            if (!used[i] && dist[i] < best) {
                best = dist[i];
                idx = i;
            }
        }
        return idx;
    }

    // w[u][v] = peso; 0 significa "sem aresta" nesta versão
    didática
    void dijkstra(int w[MAXV][MAXV], int V, int s, int dist[],
    int parent[]) {
        int used[MAXV];

        for (int i = 0; i < V; i++) {
            dist[i] = INF;
            parent[i] = -1;
            used[i] = 0;
        }
        dist[s] = 0;

        for (int iter = 0; iter < V; iter++) {
            int u = minDist(dist, used, V);
            if (u == -1) break;          // não há mais alcançáveis
            used[u] = 1;

            // Relaxamento das arestas saindo de u
            for (int v = 0; v < V; v++) {
                if (w[u][v] > 0 && !used[v]) {
                    if (dist[u] + w[u][v] < dist[v]) {
                        dist[v] = dist[u] + w[u][v];
                        parent[v] = u;
                    }
                }
            }
        }
    }
```

Código-fonte 7 - Dijkstra simples: Seleção do menor `dist[]` + relaxamento
Fonte: Elaborado pelo autor (2026)

3.2.2 Reconstrução do caminho em Dijkstra

Assim como em BFS, guardar `parent[]` permite reconstruir um caminho ao final. Esse mecanismo de predecessor é uma técnica clássica associada ao problema de menor caminho: ao armazenar o predecessor, conseguimos recuperar a sequência de vértices do destino até a origem (Black, 2020).

A mesma função de reconstrução usada em BFS pode ser reaproveitada em Dijkstra, conforme o código abaixo, porque `parent[]` tem o mesmo significado: “qual foi o último vértice antes deste no melhor caminho encontrado?”.

```
#include <stdio.h>

void printPathParent(int parent[], int s, int t) {
    int stack[MAXV], top = 0;

    for (int v = t; v != -1; v = parent[v]) {
        stack[top++] = v;
        if (v == s) break;
    }

    if (top == 0 || stack[top - 1] != s) {
        printf("Sem caminho de %d para %d.\n", s, t);
        return;
    }

    for (int i = top - 1; i >= 0; i--) {
        printf("%d%s", stack[i], (i ? " -> " : "\n"));
    }
}
```

Código-fonte 8 - Função `printPath()` (reutilizável)
Fonte: Elaborado pelo autor (2026)

3.2.3 Observação importante: Pesos negativos

Pesos negativos introduzem complicações: o significado de “melhor” caminho pode ficar instável e pode existir ciclo negativo que torna o conceito de menor caminho sem sentido. Por isso, Dijkstra é apresentado para pesos não negativos; para lidar com pesos negativos (sem ciclos negativos), uma alternativa clássica é a Bellman-Ford (Sedgewick; Wayne, 2016).

4 CONCLUSÃO

Este capítulo apresentou grafos como uma estrutura essencial para modelar redes. Aprendemos o vocabulário básico (vértices, arestas e adjacência), diferimos grafos direcionados e não direcionados e compreendemos por que pesos mudam o significado de “menor caminho”. Além disso, ficou claro que a escolha entre matriz e

lista de adjacência afeta diretamente operações centrais, especialmente a iteração sobre vizinhos, que sustenta BFS, DFS e algoritmos de caminhos (Bockholt; Black, 2021; Shukla, 2023; Sedgewick; Wayne, 2016).

Na parte algorítmica, vimos BFS como um percurso “por camadas” (fila) e DFS como um percurso “em profundidade” (recursão/pilha). Esses dois percursos formam a base para responder perguntas de conectividade, componentes e alcançabilidade em redes. Por fim, estudamos o problema de menor caminho, distinguindo o caso não ponderado (BFS) do caso ponderado (Dijkstra), reconhecendo também limitações e alternativas quando há pesos negativos (Black, 2020; Sedgewick; Wayne, 2016).

REFERÊNCIAS

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 3. ed. Cambridge, MA: The MIT Press, 2009.

IME-USP. *Estruturas de dados para representação de grafos. Algoritmos para grafos* [online]. Disponível em: https://www.ime.usp.br/~pf/algoritmos_para_grafos/aulas/graphdatastructs.html. Acesso em: 19 jan. 2026.

KERNIGHAN, B. W.; RITCHIE, D. M. *The C Programming Language*. 2. ed. Englewood Cliffs: Prentice Hall, 1988.

MOY, John. *OSPF Version 2 (RFC 2328)*. Standards Track, abr. 1998.

RADFORD UNIVERSITY. *Graphs — Terminology and Representation*. Disponível em: <https://sites.radford.edu/~nokie/classes/360/graphs-terms.html>. Acesso em: 10 jan. 2026.

RUNESTONE ACADEMY. *Implementing Breadth-First Search. Problem Solving with Algorithms and Data Structures* [online]. Disponível em: https://runestone.academy/ns/books/published/pswadsup/graphs_implementing-breadth-first-search.html. Acesso em: 19 jan. 2026

SEEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Boston: Addison-Wesley Professional, 2016.

SHUKLA, Sandeep Kumar. *Adjacency-matrix representation*. In: *Dictionary of Algorithms and Data Structures (NIST)* [online]. Ed. Paul E. Black. 8 jun. 2023.

WEISS, M. A. *Data Structures and Algorithm Analysis in C*. 2. ed. Menlo Park, CA: Addison-Wesley, 1997.